Code Logic (Feb 01) - Karan:

1. **Contest 1 Node:**
    - ROS2 node given called contest1_node
    - Job is to autonomously explore for 8 mins (480 seconds) while:
        - Avoiding obstacles using LiDAR
        - Recovering from collisions using bumpers
        - Safe speed caps
        - Detecting being stuck using odom logic

2. **ROS stuff:**
    - Publisher:
        - Publishes velocity commands on /cmd_vel using geometry_msgs/TwistStamped
        - Every cycle it sets:
            - vel.twist.linear.x = forward or back speed
            - Vel.twist.angular.z = turn speed
    - Subscribers:
        1. LiDAR scan (/scan):
            a. Computes minLaserDist_ (closest obstacle)
            b. Left_min_dist_ and right_min_dist (which side is more open)
        2. Bumpers (/hazard_detection):
            a. Used to detect if bumpers sensors are pressed
            b. Updates a map bumpers_ like “bump_left”, “bump_front_right” …
        3. Odometry (/odom)
            a. Position tracking (pos_x_, pos_y_)
            b. Also added stuck detection (to check if robot is actually moving by checking distance moved over time)
    - Timer:
        - Runs control loop every 100 ms (or 0.1 seconds) - 10Hz

3. Robot startup:

- If no odom received yet (have_odom_ is false) and if no scan received yet (have_scan_ is false)::
    - linear , angular = 0.0
    - Publish stop
    - Return
- Done so that robot never starts moving until sensors are alive.

4. Time limit logic:

- At top of control loop: (in starter code itself)
    - Elapsed time is computed from the start_time_
    - If elapsed time >= 480 sec:
        - Publishes stop command
        - Calls rclcpp::shutdown()
        - return

5. LaserCallback (LiDAR processing logic):

- Basic scan recording:
    - nLasers_
    - laserRange_
    - desiredAngle_ = 15 (set to +/-15 degrees to get 30 degrees field of view)
    - desiredNLasers_ = converts angle into number of rays (same as Tut 2)
- Finding robot "front" direction in the scan frame:
    - Tut 2 slide assumes there is a -90deg offset in LiDAR frame v/s robot forward so front_idx is the ray index that corresponds to the robot front
    - Kept the same as tut slide
- Left v/s right open space measurement:
    - Initialize left_min_dist_ and right_min_dist as infinity
    - Then look at set of rays on left and right side near front direction
    - For each ray:
        - If finite value -> valid and we keep the minimum dist on each side
        - Larger min distance means more open so fewer obstacles are close
    - Front obstacle distance (minLaserDist_):
        - Compute min distance in front (+/- 15deg) around the front_idx (same logic as Tut 2)
        - Also added start_idx and end_idx so that no undefined (underflow) or infinity (overflow) - keeps indices safe - AI suggested to do so
        - Only includes finite values
        - Gives us minLaserDist_ = closest obstacle ahead (in our FOV). This drives our main decision to move forward or turn the robot.
        -

6. HazardCallback (bumper logic):

- If hazard message will come:
    - Reset all bumper flags to false
    - Loop through detections (same as tut):
        - If detection type is BUMP: set bumpers_[frame_id] = true
        - Log which bumper is triggered
        - Gets treated like a confirmation for collision (same logic as Tut 2)

7. OdomCallback (odometry logic):

- Save pos_x_ and pos_y_
- Extract yaw_ using tf2::getYaw()
- Sets Have_odom_ = true
- Stuck detection (if robot gives message that its moving but no actual distance has been covered): https://snikolov.wordpress.com/2011/02/27/going-nowhere-fast-how-to-tell-if-your-robot-is-stuck-and-what-to-do-about-it/
    - On callback, checks last_check_time = current time
    - Last_check_x and last_check_y = current robot pose
    - Initializes progress (init_progress_ = true)
    - This sets a reference for if we the robot actually moved

8. Control Loop (main decision logic):

- Set constants (may need to tune these based on simulation and testing results):
    - Stop_Dist = 0.3m (if obstacle closer than this, then stop and turn)
    - Slow_Dist = 0.5m (if obstacle closer than this but not too close, then move slowly)
    - Fast_Speed = 0.25 m/s
    - Slow_speed = 0.10 m/s
    - Turn_speed = pi/6 (30 deg/s)

- **Bumper recovery:**
    - Detect bumper state (any_bumper_pressed)
    - If bumper is pressed and not already in recovery state:
        - Sets recovering = true and start a timer (recoverStart = now())
        - Decides which direction to turn away:
            - Left bumper hit -> turn right
            - Right bumper hit -> turn left
            - Center hit -> random left/right (coin flip logic +1, -1)
    - Recovery state:
        - For 0.6 sec: moves backwards slowly (set at linear = -0.06, ang = 0)
        - Till 1.8 sec (double of 0.6 sec = additional 1.2 sec) : turns robot in place away (linear = 0, angular = turn_speed * direction)
        - After that: end recovery, stop briefly → return to normal logic.

- **LiDAR obstacle avoidance:**
    - If obstacle too close (minLaserDist_ < Stop_dist):
        - Stop forward motion
        - Choose turn direction based on more open side:
            - If left_min_dist_ > right_min_dist → turn left
            - If left_min_dist_ < right_min_dist → turn right
            - If equal → random left or right
    - If obstacle is not too close (safe area):
        - If within slow zone (minLaserDist_ < Slow_Dist_):
            - Drive forward slowly at slow_speed (0.1 m/s)
        - Else:
            - Drive forward at fast_speed (0.25 m/s)

- **Stuck detection (odom based logic):**
    - Runs after we decide the desired motion/state
    - Checks every 2.0 sec (check_period), min. Expected movement (5cm = 0.05 m) and only triggers if intended to go forward (linear_ > want_forward (0.05m))

    - Every 2 sec: measure distance moved since last check (hypot(dx, dy)) - takes Euclidean distance
    - If robot commanded to go forward but moved less than 5cm or 0.05m:
        - Gives a warning “Stuck detected”

- Goes in forced recovery:
    - Recovering = true; recoverStart = now()
    - Random turn direction
    - Set current motion to zero in the cycle
    - So if robot is trapped in a corner or something in which the LiDR keeps asking it to move but it physically cannot, we force recovery or an escape sequence.

9. Speed caps (for safety):

- Hard linear speed cap (fixed between -0.25 and +0.25 m/s)
- Extra slow zone cap (if minLaserDist_ < Slow_Dist → fix speed between -0.1 and 0.1 m/s)
- Angular speed cap (set between -1.0 and 1.0, usual speed is pi/6 = 30 deg or 0.57 rad/s, so basically double of that, can tune)

So basically end of every controlLoop() call:

- Create TwistStamped
- Stamp with current time [now()]
- Set speeds (linear.x and angular.z) and publishes

Main function (end of given starter code):

- Added srand(time(nullptr)) so that random turns differ each run, if we don't add it will repeat the same random value generated in first run of control loop - ai helped